

سامانه OBH:

راهنمای تولید امضای دیجیتال

پاییز ۱۳۹۷

معاونت نرم افزار ۱ – مدیریت سامانه های نوین بانکداری الکترونیک

شرکت خدمات انفورماتیک

ویرایش ۰.۳

تاریخ ایجاد مستند: ۱۳۹۷/۷/۱	هاب بانکداری باز (OBH)	 شرکت خدمات انفورماتیک مدیریت سیستم های نوین بانکداری الکترونیک
طبقه بندی سند: حساس	راهنمای تولید امضای دیجیتال	

فهرست مطالب

۱. محاسبه رشته استاندارد از اطلاعات ارسالی درخواست (Canonical Request) ۳
۲. محاسبه StringToSign از مقدار Canonical Request ۵
۳. امضا کردن StringToSign محاسبه شده با کلید خصوصی ارسال کننده ۵
- ۳,۱ الگوریتم OBH1-HMAC-SHA256 ۵
- ۳,۲ الگوریتم OBH2-RSA-SHA256 ۶
۴. قرار دادن امضای محاسبه شده در هدر ۶
۵. پیوست : نحوه تولید کلید امضای دیجیتال ۷

تاریخ ایجاد مستند: ۱۳۹۷/۷/۱	هاب بانکداری باز (OBH)	 شرکت خدمات انفورماتیک مدیریت سیستم های نوین بانکداری الکترونیک
طبقه بندی سند: حساس	راهنمای تولید امضای دیجیتال	

۱. محاسبه رشته استاندارد از اطلاعات ارسالی درخواست (Canonical Request)

به منظور اطمینان از عدم تغییر اطلاعات ارسالی از سوی TPP ها در بین راه ، لازم است تمامی اطلاعات مورد نیاز که در این مستند ذکر می گردند امضا شوند. در صورتی که امضای ارسالی TPP و امضای تولید شده در OBH با یکدیگر برابر باشند می توان نتیجه گرفت اطلاعات بدون تغییر بوده اند. روش تهیه امضا در سمت ارسال کننده باید دقیقا مشابه روش تهیه امضا در سمت دریافت کننده باشد. در نتیجه باید این روش استاندارد شود. رشته ای که در نهایت امضا از آن تهیه می شود، canonical request نامیده شده است. برای ایجاد canonical request می بایست یک رشته شامل اطلاعات زیر تهیه کرد:

```

HTTPRequestMethod + '\n'
CanonicalURI + '\n'
CanonicalHeaders + '\n'
SignedHeaders + '\n'
payloadHash

```

که در آن :

۱. **HTTPRequestMethod** یکی از روشهای HTTP می باشد. به طور مثال POST یا GET
۲. **CanonicalURI** مسیر کامل دسترسی به سرویس مورد نظر است. این مقدار از اولین "/" پس از نام دامنه شروع شده و تا انتهای URI ادامه پیدا می کند. به طور مثال اگر URI به صورت زیر باشد، قسمت قرمز رنگ canonicalURI خواهد بود.

<https://obh.ir/obh/api/pisp/0100000025624/BSI/transfer-to>

۳. **CanonicalHeaders** لیستی از sign header به همراه مقادیر آنها می باشد. برای محاسبه این مقدار باید به موارد زیر توجه گردد.

- ۳،۱. هر یک از هدرها (شامل نام و مقدار آن) باید بوسیله "\n" از یکدیگر جدا شوند.
- ۳،۲. تمام حروف نام هدرها باید به صورت Lowercase باشد.
- ۳،۳. هدرها باید براساس نام آنها به صورت الفبایی مرتب شده باشند.
- ۳،۴. هر گونه space اضافی در مقادیر هدرها باید حذف شود.
- ۳،۵. \n پس از ساخت آخرین هدر نیز باید قید شود. بنابراین در انتهای ساخت canonicalHeaders دو \n خواهیم داشت. ساختار کلی ایجاد canonicalHeaders به صورت زیر است:

```

Lowercase(<HeaderName1>)+":"+Trim(<value>)+"\n"
Lowercase(<HeaderName2>)+":"+Trim(<value>)+"\n"
...
Lowercase(<HeaderNameN>)+":"+Trim(<value>)+"\n"

```

مثال:

تاریخ ایجاد مستند: ۱۳۹۷/۷/۱	هاب بانکداری باز (OBH)	 شرکت خدمات انفورماتیک مدیریت سیستم های نوین بانکداری الکترونیک
طبقه بندی سند: حساس	راهنمای تولید امضای دیجیتال	

x-obh-timestamp:1234567890123
x-obh-uuid:96c3ed35-708e-4712-8660-b3c467f47f6F

۴. **SignHeaders** شامل لیست مرتب شده بر اساس حروف الفبا از http header هایی می باشد که قرار است در canonicalHeaders مورد استفاده قرار گیرد. در محاسبه این مقدار باید موارد زیر لحاظ شود:
- ۴.۱. تمامی http header هایی که در این قسمت ذکر شده اند باید در محاسبه canonicalHeaders به کار روند.
 - ۴.۲. تمامی header name ها به صورت lower case باید ذکر شوند.
 - ۴.۳. Header name ها باید به ترتیب حروف الفبا مرتب شده باشند.
 - ۴.۴. Header name ها باید به کمک semicolon از یکدیگر جدا شوند.
- به طور مثال مقدار signHeaders در مثال قبلی برابر زیر خواهد بود:

x-obh-timestamp;x-obh-uuid

۵. **PayloadHash** در request هایی از نوع Post، بدنه درخواست نیز لازم است در canonicalRequest قید شود. json مربوطه پس از حذف double quotation (") ها و کاراکترهای space اضافی باید به صورت زیر hash شود:

Hex(Hash256(RequestPayload))

یک مثال از بدنه درخواست به صورت زیر است.

```
{destinationAccountNo=0100001676008,amount=20,destinationBank=6}
```

بدنه مربوطه سپس باید با الگوریتم SHA256 ، encrypt شود و سپس باید به HEX تبدیل شود که برای این مثال به صورت زیر خواهد بود.

1490678AB30347D5D5BFB90B129EAAFD074ACFD305748D47AE808ED1BD6FD482

مثال زیر را می توان به عنوان یک نمونه نهایی از خروجی این مرحله در نظر گرفت:

POST
/obh/api/pisp/0100000025624/BSI/transfer-to
x-obh-timestamp:1234567890123
x-obh-uuid:96c3ed35-708e-4712-8660-b3c467f47f6F

x-obh-timestamp;x-obh-uuid
1490678AB30347D5D5BFB90B129EAAFD074ACFD305748D47AE808ED1BD6FD482

تاریخ ایجاد مستند: ۱۳۹۷/۷/۱	هاب بانکداری باز (OBH)	 شرکت خدمات انفورماتیک مدیریت سیستم های نوین بانکداری الکترونیک
طبقه بندی سند: حساس	راهنمای تولید امضای دیجیتال	

۲. محاسبه StringToSign از مقدار Canonical Request

در این مرحله، StringToSign توسط الگوریتم زیر محاسبه می شود. به اینصورت که Canonical Request را به کمک الگوریتم SHA-256 encrypt کرده و سپس معادل Hex آن را بدست می آوریم.

StringToSign = Hex(Hash256(CanonicalRequest))

بعنوان مثال StringToSign محاسبه شده از روی CanonicalRequest مرحله قبل بصورت زیر خواهد بود.

**StringToSign =
2C4B8BDD1CF64B27B52B89D4F4ABCBB66E3308D2AFE9203DE94B354C500121E3**

۳. امضا کردن StringToSign محاسبه شده با کلید خصوصی ارسال کننده

در این مرحله ارسال کننده باید StringToSign بدست آمده در مرحله قبل را sign کنیم. کلید مورد استفاده برای sign رشته بسته به الگوریتم متفاوت خواهد بود. در حال حاضر دو الگوریتم برای تولید امضای دیجیتال وجود دارد: OBH1-HMAC-SHA256 و OBH2-RSA-SHA256 که در نوع کلید و الگوریتم امضا متفاوت می باشند.

۳.۱ الگوریتم OBH1-HMAC-SHA256

در این الگوریتم کلید خصوصی کاربر ترکیبی است از سال میلادی جاری بعلاوه clientID و clientSecret که بصورت زیر می باشد:

Key = SHA256(YEAR+ClientID+ClientSecret)

بعنوان مثال برای یک TPP با clientID=IUCBvuxOJa و clientSecret=w5mqUq46hZ کلید بصورت زیر خواهد بود

SHA256(2018IUCBvuxOJaW5mqUq46hZ)

پس از محاسبه کلید، canonicalRequest که در مرحله قبل hash شده است به صورت زیر امضا می شود:

Signature = Hex(HmacSHA256(key, StringToSign))

مثال زیر را می توان به عنوان یک نمونه نهایی امضا در نظر گرفت:

646002FAC8B316E8AE2CD63F8203082B54F7293839D17038394E98BB8174FFF4

تاریخ ایجاد مستند: ۱۳۹۷/۷/۱	هاب بانکداری باز (OBH)	 شرکت خدمات انفورماتیک مدیریت سیستم های نوین بانکداری الکترونیک
طبقه بندی سند: حساس	راهنمای تولید امضای دیجیتال	

۳.۲ الگوریتم OBH2-RSA-SHA256

اما الگوریتم دوم RSA بوده و با یک کلید خصوصی معتبر با گواهی نماد امضا صورت می گیرد. نحوه تولید این کلید در پیوست آمده است.

```
Signature sig = Signature.getInstance("SHA256withRSA");
sig.initSign(privateKey);
sig.update(stringToSign.getBytes("UTF-8"));
Signature = Base64.getEncoder().encodeToString(sig.sign());
```

۴. قرار دادن امضای محاسبه شده در هدر

پس از محاسبه امضا که در مراحل قبل شرح داده شد، لازم است که امضای تولید شده با فرمت زیر در هدر X-Obh-signature قرار گرفته و ارسال گردد. X-Obh-signature شامل سه بخش است که بوسیله semicolon از هم جدا شده اند. بخش اول الگوریتم تولید امضا است که شامل دو الگوریتم OBH1-HMAC-SHA256 و OBH2-RSA-SHA256 می باشد. بخش دوم signHeaders=Y می باشد که در آن Y هدرهایی است که در تولید امضا به کار رفته اند و توسط کاما از هم جدا شده اند و نهایتاً بخش سوم signature=X بوده که در آن X امضای تولید شده در مراحل قبل می باشد.

```
X-Obh-signature= OBH1-HMAC-SHA256 ;signedHeaders=Y ;signature=X
```

در این مثال، هدر نهایتاً به صورت زیر خواهد بود.

```
OBH1-HMAC-SHA256;SignedHeaders=x-obh-uuid,x-obh-timestamp
;Signature=646002FAC8B316E8AE2CD63F8203082B54F7293839D17038394E98BB8174FF
F4
```

تاریخ ایجاد مستند: ۱۳۹۷/۷/۱	هاب بانکداری باز (OBH)	 شرکت خدمات انفورماتیک مدیریت سیستم های نوین بانکداری الکترونیک
طبقه بندی سند: حساس	راهنمای تولید امضای دیجیتال	

۵. پیوست : نحوه تولید کلید امضای دیجیتال

هر TPP باید یک CSR برای دریافت گواهی برای نماد ارسال کند. اقلام موجود در این CSR به شرح زیر می باشد.

مقدار	قلم اطلاعاتی	
"ir"	<c>	اطلاعات Subject
آدرس URI مربوط به سیستم	<cn>	
نام مسئول گواهی (به فارسی و مطابق شناسنامه)	<g>	
نام خانوادگی مسئول گواهی (به فارسی و مطابق شناسنامه)	<sn>	
شناسه شهاب 16 رقمی مسئول گواهی	<serialnumber>	
آدرس پست الکترونیک مسئول گواهی	<e>	
نام سازمان + "-" + شناسه شهاب 16 رقمی سازمان	<o>	
Digital Signature, Key Encipherment	کاربرد کلید (Key Usage)	
Client Authentication, Server Authentication	کاربرد کلید الحاقی (Extended Key Usage)	
مقدار کلید RSA 2048 بیت	کلید عمومی (Public Key)	

برای تولید CSR ابتدا فایل کانفیگ sysCustomers.cfg را که بصورت زیر می باشد تهیه و ویرایش کنید.

```
#####
## CUSTOMERS SYSTEM CERTIFICATE SUBJECT INFO ##
#####
orgURL = www.rgtamin.ir          # Domain's IP or URL
orgName = اجتماعی تامین گستر رفاه  # Organization Name in Farsi
orgShahab = 2000010101775377      # Organization SHAHAB ID – 16 digits

personName = رضا                # System Admin. Name in Farsi
personFamily = خادم             # System Admin. Family (surname) in Farsi
personShahab = 1000000349017611  # System Admin. SHAHAB ID - 16 digits
personEmail = rkhadem@rgtamin.ir # System Admin. Email Address

#####
## DO NOT CHANGE LINES BELOW ##
#####
[ req ]
default_bits = 2048
default_md = sha256
prompt = no
distinguished_name = req_distinguished_name
req_extensions = req_ext

[ req_distinguished_name ]
countryName = "ir"
```

تاریخ ایجاد مستند: ۱۳۹۷/۷/۱	هاب بانکداری باز (OBH)	 شرکت خدمات انفورماتیک مدیریت سیستم های نوین بانکداری الکترونیک
طبقه بندی سند: حساس	راهنمای تولید امضای دیجیتال	

```

organizationName = $ENV::orgName-$ENV::orgShahab
commonName = $ENV::orgURL
emailAddress = $ENV::personEmail
givenName = $ENV::personName
surname = $ENV::personFamily
serialNumber = $ENV::personShahab

```

```

[ req_ext ]
subjectAltName = @alt_names

```

```

[ alt_names ]
DNS.1 = $ENV::orgURL

```

سپس با کمک دستور زیر کلید و CSR را تولید کنید.

```

openssl req -config sysCustomers.cfg -newkey rsa:2048 -keyout "keyPrivate.key" -out "certRequest.csr" -utf8

```

پس از آن CSR ایجاد شده را به همراه فرمهای مربوطه به نماینده نماد ارسال کرده و پس از دریافت گواهی مربوطه (**orgCertificate.cer**) از نماد آن را به **orgCertificate.der** تغییر نام داده و مانند زیر در کلید خصوصی ایجاد شده (**keyPrivate.key**) تزریق کند.

```

openssl pkcs12 -export -out signature.p12 -inkey keyPrivate.key -in orgCertificate.der -certfile customersTrustChain.cer

```

کلید **signature.p12** باید در تهیه امضای دیجیتال نسخه ۲ توسط TPP ها به کار رود. علاوه بر این گواهی مربوطه باید جهت تایید صحت امضای دیجیتال ارسالی، در اختیار شاهین قرار گیرد.